Projet gestionyobs

Logiciel Windows de gestion pour quincaillerie (Flutter + SQLite)

Récapitulatif de l'avancement, de l'architecture et des décisions, pour reprendre le travail plus tard.

1. Objectif du projet

Créer un logiciel Windows (fichier .exe) avec Flutter, installé dans une quincaillerie pour suivre ses activités : articles, ventes, stock, achats, clients, caisse. On a choisi de commencer par le **cas 1 : base SQLite locale** (un seul ordinateur, sans internet). Les cas suivants (serveur local, API distante, synchronisation) viendront ensuite, en réutilisant la même interface.

2. Environnement et packages

- Windows 10, Flutter (canal stable), Visual Studio avec la charge de travail « Développement Desktop en C++ ».
- Projet : **gestionyobs** (dossier D:\gestionyobs). Lancement : `flutter run -d windows`.
- Logo : assets/images/logo.jpg, déclaré dans pubspec.yaml sous `assets:`.

Package	Rôle
path_provider	Donne les dossiers où l'application peut écrire (AppData, Documents)
sqflite_common_ffi	Permet d'utiliser SQLite sur Windows
sqlite3_flutter_libs	Embarque le moteur SQLite dans le .exe
bcrypt	Hache les mots de passe (ils ne sont jamais enregistrés en clair)

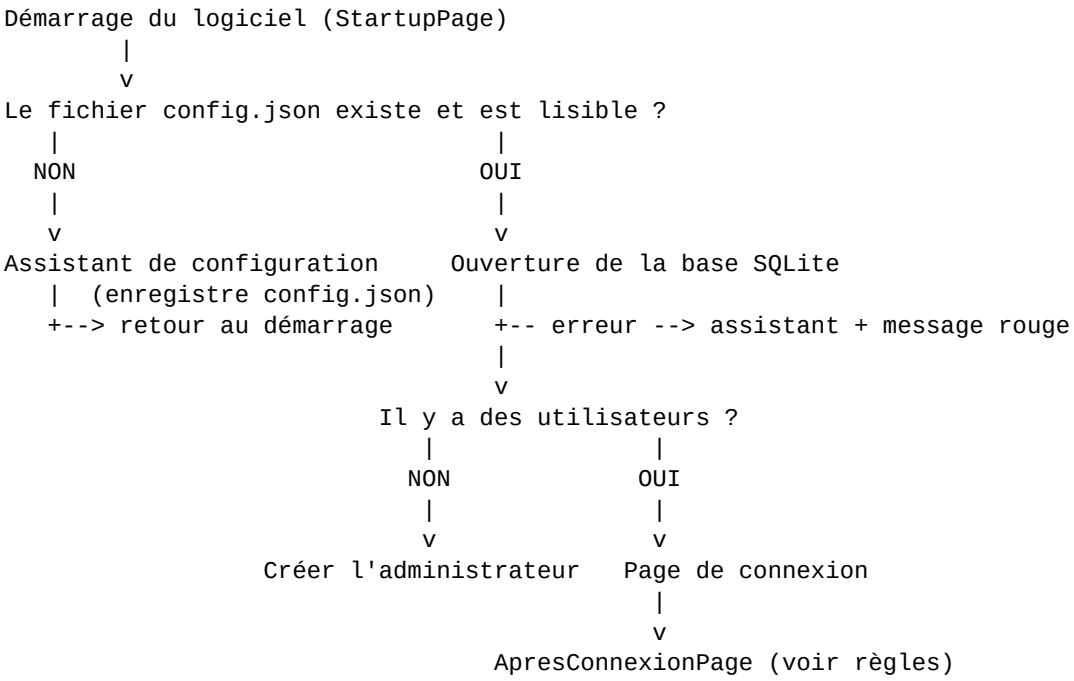
Ajout d'un package : `flutter pub add nom_du_package` (dans le dossier du projet).

3. Structure du code (dossier lib/)

Fichier	Rôle
main.dart	Point d'entrée ; thème vert ; démarre sur StartupPage
startup_page.dart	Au démarrage : lit la configuration, ouvre la base, décide de la première page
config_service.dart	Modèle AppConfig et lecture / écriture / suppression de config.json
config_page.dart	Assistant de configuration (type de base, emplacement de la base, dossier des sauvegardes)
database_service.dart	Ouverture de la base SQLite, création et migration des tables, requêtes

Fichier	Rôle
utilisateur.dart	Modèles Utilisateur et Agence, et Session (utilisateur et agence courants)
premier_admin_page.dart	Création du premier compte administrateur
login_page.dart	Page de connexion (logo, identifiant, mot de passe avec oeil, validation)
apres_connexion_page.dart	Après la connexion : décide entre création d'agence, choix d'agence ou accueil
creer_agence_page.dart	Formulaire de création d'agence (nom, ville, téléphone, description)
choix_agence_page.dart	Liste des agences accessibles ; bouton « Nouvelle agence » pour l'admin
home_page.dart	Accueil provisoire : utilisateur, rôle, agence ; boutons changer d'agence et déconnexion

4. Déroulement au démarrage



5. Fichier de configuration

- Format JSON, nommé **config.json**, dans le dossier de données de l'application (AppData, via path_provider). Il ne faut pas le placer à côté du .exe, car Program Files est protégé en écriture.
- Contenu pour SQLite : type, cheminBase (emplacement du fichier .db), dossierSauvegarde. Pas de login ni de mot de passe : SQLite est un simple fichier.
- Le login et le mot de passe de base serviront pour le **cas 2** (MySQL / PostgreSQL), avec un stockage sécurisé du mot de passe (flutter_secure_storage), pas dans le JSON.

- Si le fichier est absent ou abîmé, ou si la base est introuvable, on revient à l'assistant avec un message clair, sans plantage.

```
{
  "type": "sqlite",
  "cheminBase": "C:\\...\\quincaillerie.db",
  "dossierSauvegarde": "D:\\Sauvegardes"
}
```

6. Base de données

Versions de la structure

Version	Contenu
1	Table utilisateurs
2	Tables agences et utilisateur_agences
3	Colonne description dans agences

Tables

Table	Colonnes principales
utilisateurs	id, nom_complet, identifiant (unique), mot_de_passe_hash, role (admin ou vendeur), actif, cree_le
agences	id, nom (unique), ville, telephone, description, actif, cree_le
utilisateur_agences	utilisateur_id, agence_id (clé primaire double, suppression en cascade) : qui travaille dans quelle agence

Règle de migration

Chaque changement de structure suit le même schéma : on augmente `_versionBase`, on met à jour le CREATE TABLE (pour les bases neuves) et on ajoute dans `onUpgrade` une migration qui **vérifie avant d'agir** (par exemple avec _colonneExiste), pour ne jamais échouer sur une base déjà à jour.

> Les requêtes utilisent toujours des paramètres (le signe ?) et jamais du texte collé dans le SQL, pour éviter les injections SQL.

7. Règles des rôles et des agences

Cas	Comportement
Aucune agence, utilisateur **admin**	Page « Créer la première agence », puis accueil dans cette agence
Admin, des agences existent	Il voit **toutes** les agences et peut en créer une nouvelle
Vendeur affilié à plusieurs agences	Il voit seulement les siennes et en choisit une
Vendeur affilié à **une seule** agence	Il entre directement dans cette agence

Cas	Comportement
Vendeur affilié à **aucune** agence	Message « Contactez l'administrateur »

Les données ne dépendent pas seulement du rôle : chaque utilisateur est affilié à une ou plusieurs agences. Plus tard, les tables des articles, du stock et des ventes auront une colonne agence_id pour que chaque agence ait ses propres données.

8. Étapes réalisées

Étape	Contenu	État
Préparation	Installation de Flutter et Visual Studio, projet gestionyobs, logo YOBS	Validé
Page de connexion	Carte, logo, champs, bouton, validation des champs vides, oeil du mot de passe	Validé
1	Service de configuration : lire, écrire, supprimer config.json	Validé
2	Assistant de configuration (SQLite, chemin de la base, sauvegardes)	Validé
3	Logique de démarrage : fichier présent, connexion ; absent, assistant	Validé
4	Base SQLite, table utilisateurs, premier administrateur	Validé
5	Connexion réelle (vérification du hash), session, accueil	Validé
6	Agences : création, choix, accès selon le rôle, description (textarea)	En cours de test

Point en attente pour l'étape 6 : une erreur « duplicate column name: description » est apparue au démarrage. La correction (migration qui vérifie l'existence de la colonne avec _colonneExiste) est dans le code final plus bas ; il reste à relancer le logiciel pour confirmer que tout démarre et à refaire les tests de la zone de description.

9. Problèmes rencontrés et solutions

Problème	Solution
`flutter pud add` : commande introuvable	Faute de frappe : la commande est `flutter pub add`
« No pubspec.yaml file found »	Le terminal n'était pas dans le dossier du projet : faire cd gestionyobs
« Lookup failed: title in @getters in MyHomePage »	Rechargement à chaud impossible après un gros changement : arrêter avec q et relancer flutter run -d windows
« Unable to load asset »	Le chemin de l'image doit être identique dans pubspec.yaml et dans Image.asset, avec la bonne extension (.jpg ici)
Une image ou un package ajouté n'apparaît pas	Les assets et les packages natifs demandent un redémarrage complet (q puis flutter run), pas un simple r
« duplicate column name: description »	Migration qui teste l'existence de la colonne avant ALTER TABLE ADD COLUMN

10. Commandes utiles

Commande	Effet
`flutter run -d windows`	Lance l'application sur Windows
`r` (dans le terminal)	Rechargement à chaud (petites modifications)
`R` (majuscule)	Redémarrage de l'application
`q`	Arrête l'application
`flutter clean`	Supprime l'ancienne compilation (utile après une erreur de build)
`flutter pub get`	Télécharge les packages du pubspec.yaml
`flutter build windows`	Génère l'exécutable dans build\windows\x64\runner\Release
`flutter doctor`	Vérifie l'installation de Flutter

11. Éléments provisoires à retirer ou changer

- Le bouton **« Reconfigurer (test) »** de la page de connexion : à retirer ou à réserver à l'administrateur.
- Le titre **« Gestion Provenderie »** (page de connexion et main.dart) : à remplacer par le nom du logiciel de quincaillerie.
- La page d'accueil est provisoire : elle sera remplacée par un menu adapté au rôle (l'administrateur voit tout, le vendeur voit la caisse).

12. Code final des deux fichiers les plus modifiés

utilisateur.dart

```
/// Un utilisateur du logiciel.
class Utilisateur {
  final int id;
  final String nomComplet;
  final String identifiant;
  final String role; // 'admin' ou 'vendeur'

  const Utilisateur({
    required this.id,
    required this.nomComplet,
    required this.identifiant,
    required this.role,
  });

  bool get estAdmin => role == 'admin';
}

/// Une agence (un point de vente).
class Agence {
  final int id;
  final String nom;
  final String? ville;
  final String? telephone;
  final String? description;

  const Agence({
```

```
    required this.id,
    required this.nom,
    this.ville,
    this.telephone,
    this.description,
  });

  factory Agence.fromMap(Map<String, Object?> m) => Agence(
        id: m['id'] as int,
        nom: m['nom'] as String,
        ville: m['ville'] as String?,
        telephone: m['telephone'] as String?,
        description: m['description'] as String?,
      );
}

/// Retient l'utilisateur et l'agence choisie tant que le logiciel est ouvert.
class Session {
  static Utilisateur? utilisateurCourant;
  static Agence? agenceCourante;

  static void deconnecter() {
    utilisateurCourant = null;
    agenceCourante = null;
  }
}
```

database_service.dart

```
import 'dart:io';
import 'package:bcrypt/bcrypt.dart';
import 'package:sqflite_common_ffi/sqflite_ffi.dart';
import 'config_service.dart';
import 'utilisateur.dart';

/// Gère la base de données SQLite.
class DatabaseService {
  static Database? _db;
  static bool _ffiInitialise = false;

  // Version 1 : utilisateurs. Version 2 : agences + affiliations.
  // Version 3 : colonne description dans agences.
  static const int _versionBase = 3;

  static Future<void> ouvrir(AppConfig config) async {
    if (config.type != 'sqlite') {
      throw Exception('Type de base non pris en charge : ${config.type}');
    }

    if (!_ffiInitialise) {
      sqfliteFfiInit();
      _ffiInitialise = true;
    }

    final dossier = File(config.cheminBase).parent;
    await dossier.create(recursive: true);

    _db = await databaseFactoryFfi.openDatabase(
      config.cheminBase,
      options: OpenDatabaseOptions(
        version: _versionBase,
        onConfigure: (db) async {
          await db.execute('PRAGMA foreign_keys = ON');
        },
        onCreate: (db, version) async {
          await _creerTableUtilisateurs(db);
          await _creerTablesAgences(db);
        },
        onUpgrade: (db, ancienneVersion, nouvelleVersion) async {
          if (ancienneVersion < 2) {
            await _creerTablesAgences(db);
          }
          if (!await _colonneExiste(db, 'agences', 'description')) {
            await db.execute('ALTER TABLE agences ADD COLUMN description TEXT');
```

```
        }
      },
    ),
  );
}

static Future<void> _creerTableUtilisateurs(Database db) async {
  await db.execute('''
    CREATE TABLE utilisateurs (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      nom_complet TEXT NOT NULL,
      identifiant TEXT NOT NULL UNIQUE,
      mot_de_passe_hash TEXT NOT NULL,
      role TEXT NOT NULL DEFAULT 'vendeur',
      actif INTEGER NOT NULL DEFAULT 1,
      cree_le TEXT NOT NULL
    )
  ''');
}

static Future<void> _creerTablesAgences(Database db) async {
  await db.execute('''
    CREATE TABLE agences (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      nom TEXT NOT NULL UNIQUE,
      ville TEXT,
      telephone TEXT,
      description TEXT,
      actif INTEGER NOT NULL DEFAULT 1,
      cree_le TEXT NOT NULL
    )
  ''');

  await db.execute('''
    CREATE TABLE utilisateur_agences (
      utilisateur_id INTEGER NOT NULL,
      agence_id INTEGER NOT NULL,
      PRIMARY KEY (utilisateur_id, agence_id),
      FOREIGN KEY (utilisateur_id) REFERENCES utilisateurs(id) ON DELETE CASCADE,
      FOREIGN KEY (agence_id) REFERENCES agences(id) ON DELETE CASCADE
    )
  ''');
}

// Vérifie si une colonne existe déjà dans une table
static Future<bool> _colonneExiste(
  Database db,
  String table,
  String colonne,
) async {
  final infos = await db.rawQuery('PRAGMA table_info($table)');
  return infos.any((c) => c['name'] == colonne);
}

static Database get db {
  final d = _db;
  if (d == null) throw StateError("La base de données n'est pas ouverte");
  return d;
}

// ---------- Utilisateurs ----------

static Future<bool> aDesUtilisateurs() async {
  final resultat = await db.rawQuery('SELECT COUNT(*) AS n FROM utilisateurs');
  final n = resultat.first['n'] as int;
  return n > 0;
}

static Future<void> creerUtilisateur({
  required String nomComplet,
  required String identifiant,
  required String motDePasse,
  required String role,
}) async {
```

```
  final hash = BCrypt.hashpw(motDePasse, BCrypt.gensalt());

  await db.insert('utilisateurs', {
    'nom_complet': nomComplet,
    'identifiant': identifiant,
    'mot_de_passe_hash': hash,
    'role': role,
    'actif': 1,
    'cree_le': DateTime.now().toIso8601String(),
  });
}

static Future<Utilisateur?> verifierConnexion(
  String identifiant,
  String motDePasse,
) async {
  final lignes = await db.query(
    'utilisateurs',
    where: 'identifiant = ? AND actif = 1',
    whereArgs: [identifiant],
    limit: 1,
  );

  if (lignes.isEmpty) return null;

  final ligne = lignes.first;
  final hash = ligne['mot_de_passe_hash'] as String;

  if (!BCrypt.checkpw(motDePasse, hash)) return null;

  return Utilisateur(
    id: ligne['id'] as int,
    nomComplet: ligne['nom_complet'] as String,
    identifiant: ligne['identifiant'] as String,
    role: ligne['role'] as String,
  );
}

// ---------- Agences ----------

// L'admin voit toutes les agences, les autres seulement les leurs.
static Future<List<Agence>> agencesDisponibles(Utilisateur utilisateur) async {
  final List<Map<String, Object?>> lignes;

  if (utilisateur.estAdmin) {
    lignes = await db.query('agences', where: 'actif = 1', orderBy: 'nom');
  } else {
    lignes = await db.rawQuery('''
      SELECT a.* FROM agences a
      INNER JOIN utilisateur_agences ua ON ua.agence_id = a.id
      WHERE ua.utilisateur_id = ? AND a.actif = 1
      ORDER BY a.nom
    ''', [utilisateur.id]);
  }

  return lignes.map(Agence.fromMap).toList();
}

// Transforme un texte vide en null (rien à enregistrer)
static String? _videEnNull(String? texte) =>
    (texte == null || texte.trim().isEmpty) ? null : texte.trim();

static Future<Agence> creerAgence({
  required String nom,
  String? ville,
  String? telephone,
  String? description,
}) async {
  final villeNettoyee = _videEnNull(ville);
  final telNettoye = _videEnNull(telephone);
  final descriptionNettoyee = _videEnNull(description);

  final id = await db.insert('agences', {
    'nom': nom,
```

```
      'ville': villeNettoyee,
      'telephone': telNettoye,
      'description': descriptionNettoyee,
      'actif': 1,
      'cree_le': DateTime.now().toIso8601String(),
    });

    return Agence(
      id: id,
      nom: nom,
      ville: villeNettoyee,
      telephone: telNettoye,
      description: descriptionNettoyee,
    );
  }
}
```

13. Prochaines étapes

Étape 7 : gestion des utilisateurs et affiliation aux agences

- Page de liste des utilisateurs (réservée à l'administrateur).
- Création d'un vendeur : nom, identifiant, mot de passe, et cases à cocher pour les agences où il travaille (remplit la table utilisateur_agences).
- Tests de tous les cas du tableau des rôles : vendeur avec une agence, plusieurs agences, aucune agence.

Ensuite

- Menu principal adapté au rôle (administrateur, vendeur) et à l'agence courante.
- Table des articles, avec stock par agence (colonne agence_id).
- Ventes et caisse, achats et fournisseurs, clients et crédits, factures et devis.
- Sauvegardes automatiques de la base vers le dossier configuré.
- Cas 2 (serveur local), cas 3 (API distante) et cas 4 (synchronisation) : à configurer plus tard.

Questions en suspens

- Le **catalogue d'articles** est-il commun à toutes les agences (mêmes produits et mêmes prix, avec un stock séparé par agence), ou chaque agence a-t-elle ses propres articles et ses prix ?
- Nom définitif du logiciel (en remplacement de « Gestion Provenderie »).